Arabian Journal for Science and Engineering (2020) 45:10019–10032
https://doi.org/10.1007/s13369-020-04552-0

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

Generating True Random Numbers Based on Multicore CPU Using Race Conditions and Chaotic Maps

Je Sen Teh[1] · Moatsum Alawida[1] · Azman Samsudin[1]

Received: 3 December 2019 / Accepted: 16 April 2020 / Published online: 14 May 2020
© King Fahd University of Petroleum & Minerals 2020

Abstract
A true random number generator (TRNG) is proposed, harvesting entropy from multicore CPUs to generate non-deterministic outputs. The entropy source is the unpredictable sequence of thread access when parallel threads attempt to access the same memory location, known as race condition or data races. Although prior work using the same entropy source exists, they either have low efficiency or insufficient security analysis. The novelty of this work lies in its use of chaotic networks capable of extracting entropy while postprocessing outputs simultaneously. These networks are formulated by coupling chaotic maps in the form of chaotic coupled map lattices which have the capability to amplify minor uncertainties, leading to better performance as compared to other CPU-based TRNGs. We first perform experiments to depict the unpredictable nature of thread access due to race conditions through entropy and scale index analysis. Next, the proposed generator is scrutinized based on a standardized set of evaluation criteria which includes the use of multiple statistical test suites followed by an analysis of its non-deterministic property. We also perform an in-depth entropy analysis of the generator's outputs and measure its degree of non-periodicity. Results indicate that the proposed chaos-based TRNG is fast, evenly distributed, and is secure enough for applications that have high security requirements.

Keywords Chaotic map · Cryptography · Entropy · Multicore CPU · Parallel processing · Random number

1 Introduction

Random numbers are utilized in various fields such as cryptography, random sampling and Monte Carlo simulations. There are two categories of random number generators: A pseudorandom number generator (PRNG) produces the same set of random numbers for a given input seed, whereas a true random number generator (TRNG) produces unpredictable random number sequences that never repeat. Most TRNGs rely on randomness from physical sources such as electronic noise [1], radioactive decay [2] and lasers [3,4], but there are also algorithms that harvest entropy from computer hardware such as DRAM [5], processor schedulers [6], graphic processing units (GPU) [7,8], microphones [9] and also mobile phone cameras [10]. Another proposed TRNG took advantage of 'social noise,' generating random numbers based on messages from Twitter [11]. The entropy of these sources is quantified, and a postprocessing algorithm is usually applied to eliminate bias. TRNGs are important for applications that require high security because their number sequences cannot be reproduced. Most of these highly sensitive applications are related to cryptography, specifically to generate nonces, one-time pads, initialization vectors, generating session keys, signature key pairs or signature parameters [12]. However, they are generally slower than their PRNG counterparts.

Multicore CPUs are widely available in modern computers and mobile devices, making them an easily accessible entropy source. Some CPU statistics such as the number of CPU cycles have been used as a seed for PRNGs to produce unpredictable outputs [13]. Although the seed itself may be non-deterministic, the underlying generator is still not considered a TRNG because the resulting sequence will always be the same for a given seed value. A truly non-deterministic generator will produce different outputs even when all the initial conditions are exactly the same. The goal of this paper is to utilize the race condition in multicore CPUs (which is regarded as an undesirable phenomena in parallel computation) to design a truly non-deterministic generator that fulfills all the requirements for high security applications. Previously

✉ Je Sen Teh
jesen_teh@usm.my

[1] School of Computer Sciences, Universiti Sains Malaysia, Gelugor, Penang, Malaysia

proposed CPU TRNGs based on race conditions suffer from low efficiency [14] or lack sufficient security evaluation to ensure their suitability for practical applications [15,16]. The majority of these CPU-based TRNGs do not analyze their outputs for non-determinism but only rely on black box statistical testing [13,15,16]. Thus, the proposed algorithm is designed to overcome these drawbacks.

To achieve this goal, entropy is harvested from multicore CPUs and any uncertainties are amplified with the help of highly sensitive dynamical systems known as chaotic maps. We first experimentally depict the non-deterministic and non-periodic nature of thread access through entropy and scale index analysis. Then proposed generator is evaluated in two stages to ensure that it has sufficient security margins: First, multiple statistical test suites are used to uncover any statistical defects of the proposed generator in addition to in-depth entropy analysis. Next, the TRNG is analyzed for non-determinism, non-periodicity, followed by evaluation of its random number distribution. Experiments are performed on a dual-core (Intel Core2Duo, 2.0 GHz, 3 GB RAM) and quad-core (Intel Core i7-4700MQ, 2.4 GHz, 8 GB RAM) computer. The algorithm is written in C++, utilizing OpenMP for parallel execution.

The rest of this paper is structured as follows: Sect. 2 provides details on how multicore architectures can be utilized as an entropy source, followed by an introduction to chaotic maps. Section 3 describes the implementation of the proposed TRNG based on multicore CPU architectures. Analysis of race condition thread access, security and performance evaluation is discussed in Sects. 4, 5 and 6, respectively. Finally the results of this paper are summarized in Sect. 7.

2 Preliminaries

2.1 CPU Randomness

In a parallel computing environment, multiple threads are spawned to execute instructions. When these threads attempt to read or write to the same memory location, the order in which these threads are granted access is non-deterministic unless they are forcefully serialized or synchronized by the program. This phenomenon is known as a race condition which can give rise to non-deterministic parallel programs [17,18]. Race conditions exist due to hardware and software influences. One example is instruction pipelining where multiple instructions are split into smaller parts to be computed in parallel. Background processes and tasks affect how these pipelines are filled for individual CPUs. This causes every CPU to compute even the same task at different speeds. In addition, CPUs operate faster than memory which causes arbitrary waiting times when multiple CPUs attempt to access the same memory segment. Other sources of CPU randomness include CPU frequency scaling, cache misses and timer interrupts [15]. We experimentally analyze the unpredictability of thread access in Sect. 4.

Implementations of TRNGs based on CPU race conditions have not been the subject of in-depth study in recent literature. Colesa et al. introduced a software TRNG that exploits race conditions to generate random numbers that can pass the NIST statistical test suite [15]. They also discussed the various hardware influences that cause the non-deterministic nature of thread access to a shared variable. Kim et al. proposed another TRNG based on multicore CPUs using OpenMP for thread control [16]. Their generator was evaluated by measuring the minimum entropy of the generated sequences without any statistical testing. Neither of these proposals analyzed the non-deterministic properties of the TRNG. Muller proposed a TRNG with an entropy source based on CPU execution time jitter [14]. It was extensively analyzed using entropy analysis as well as statistical test suites. Unlike the previous implementations which did not record any generation speed, Muller's proposal achieved a throughput of up to 80 Kbps on an Intel Core i7.

Based on the review of previous work, it was found that previously proposed CPU-TRNGs either lacked in security evaluation or are inefficient. With this in mind, we propose a new CPU-TRNG that is both fast and has thorough security evaluation. The design and evaluation results can be used as a benchmark for future CPU-based TRNGs. To achieve these goals, chaotic maps will be utilized to amplify uncertainty as well as for postprocessing.

2.2 Chaos Theory

The number of CPU cores available in most personal computers (PC) is limited to two to four cores. Therefore the number of competing threads is relatively low which can lead to insufficient uncertainty to generate statistically sound random numbers. In order to amplify this minor uncertainty, chaotic maps are used in the design of the proposed algorithm. Chaotic maps are dynamical systems which have high sensitivity to slight changes to their initial conditions and control parameters. Chaotic maps also have pseudorandom behavior, ergodicity, as well as diffusion and confusion characteristics. These properties are attractive in the field of cryptography, which led to the proposal of many chaos-based cryptosystems such as encryption algorithms [19,20] and hash functions [21]. Chaos has also been used in the design of random number generators [9,10] and as a postprocessing function for TRNGs [22]. To achieve good performance and security, the proposed algorithm utilizes the chaotic logistic map which is defined as

$$C(x) = rx(1 - x), \tag{1}$$

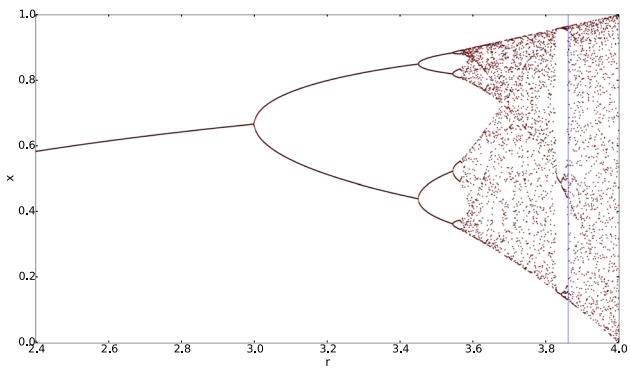

Fig. 1 Logistic map bifurcation diagram [41]

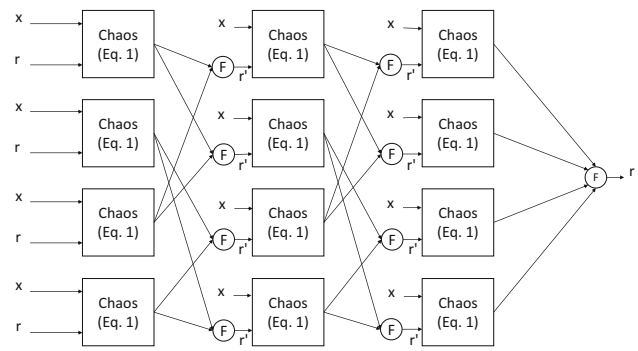

Fig. 2 Chaotic Omega network

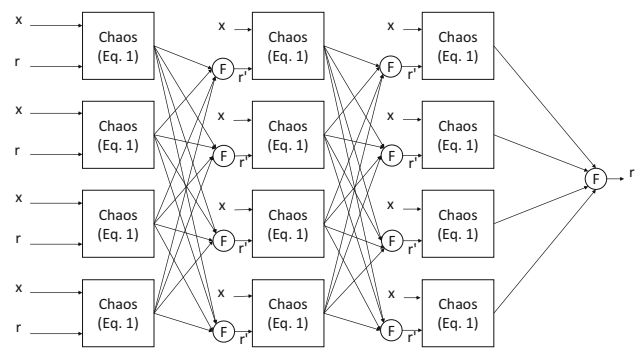

Fig. 3 Chaotic Clos network

where x is the chaotic state and r is a control parameter that affects the behavior of the chaotic map. Out of the various chaotic maps available, the logistic map was selected as a proof of concept due to its simplicity and sufficient chaoticity with the proper selection of r. However, for the purpose of random number generation, any other chaotic map can be used in its place as long as they operate within their chaotic range. In addition, it is also one of the most widely studied chaotic maps for cryptographic purposes. When r is within the range of $[3.86, 4]$, the logistic map will depict highly chaotic outputs where a slight change to x or r will lead to a large change to $C(x)$. This region of operation can be observed in the logistic map's bifurcation diagram as shown in Fig. 1, whereby the vertical line indicates the value of $r = 3.86$. Prior to this region, the logistic map has periodic behavior or has windows of periodicity which is undesirable when designing RNGs or other cryptographic algorithms.

The logistic map is an iterative function, where x will be updated to a new value after each iteration. In the rest of this paper, the term *output* of the logistic map refers to the new value of x that is updated after each iteration.

3 Algorithm Description

In the proposed TRNG, CPU threads are forced to update the same shared x and r value for multiple instances of the logistic map. The order in which these threads access these variables is unpredictable as explained in Sect. 2.1. A slight change made by one thread to the x or r value will affect the logistic map calculation of another, leading to an avalanche effect.

To further enhance this avalanche effect, the logistic maps are arranged based on the structure of two chaotic coupled map lattices (CCML), which are denoted as the 8×8 Omega network and the 8×8 Clos network as depicted in Figs. 2 and 3. The naming of these networks is based on the Omega and Square 3-Stage Clos shuffle-exchange networks which inspired the mapping patterns between stages. The minimum number of stages, β required for full diffusion (one input propagating to all outputs), is three and two for the Omega and Clos networks as shown in Fig. 4. To ensure that the computational time is not dependent on the type of network being chosen, both networks are set to three stages. This ensures higher resistance against timing attacks. The use of two different chaotic networks amplifies any unpredictable behavior threefold: once by the sensitivity of the chaotic map itself, then by the diffusion property of the network and finally by the slight mapping difference between the Omega and Clos network.

When the algorithm is first executed, the shared x variable is set to 0.36089632 which consists of arbitrary random numbers taken from pi, whereas the default value of shared r is set to 4.0. Any arbitrary value can be selected for x with the exception of $x = 0$. The IEEE 754-2008 standard for floating numbers is used in the proposed algorithm. Every CPU thread is assigned one network to compute based on the current value stored in the shared x variable. If $x \geq 0.5$, the Omega network is selected, whereas the Clos network is selected for $x < 0.5$. Shared x is updated while iterating each chaotic map, whereas the shared r is only accessed at the beginning of the network and at the end. During each iteration, the shared x variable is normalized as

$$\hat{x} = 1000x - \lfloor 1000x \rfloor, \tag{2}$$

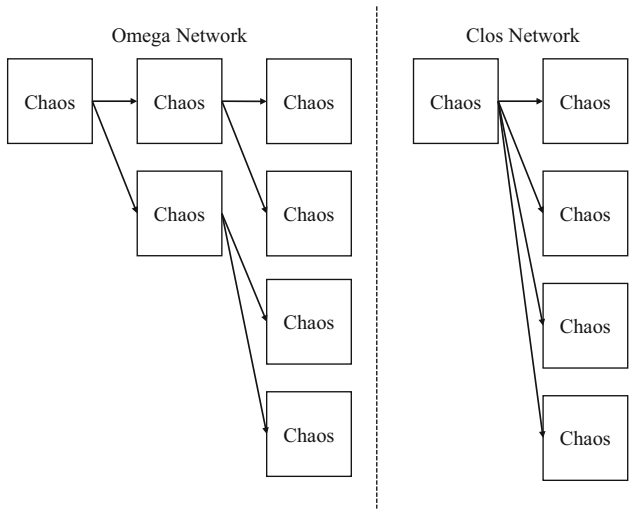

Fig. 4 Comparison of diffusion properties

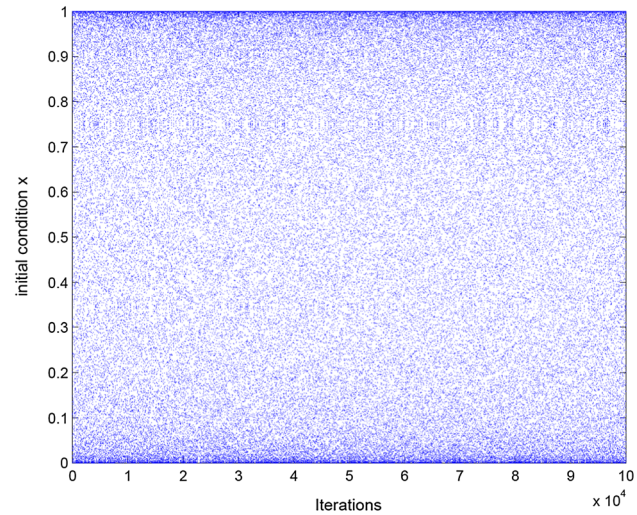

Fig. 5 Dist. of x before normalization

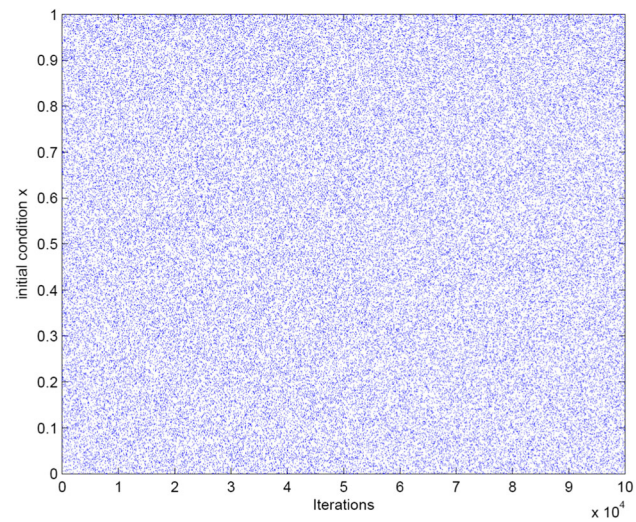

Fig. 6 Dist. of x after normalization

to improve its complexity and uniformity of distribution [23]. Without this modification, the values of x after each iteration tend to move along the logistic map's expected phase space. By using Eq. 2, the logistic map is forced to start the sequence from different points in the phase space every round which changes the unimodal nature of the chaotic map to a more complex one. In addition, changes to the three significant digits of two consecutive x values in a trajectory can be low, whereas the probability of change for the second three significant digits is higher (e.g., $0.740\mathbf{9}36 \rightarrow 0.740\mathbf{8}36$ when $r = 3.86$). Therefore, three most significant digits from the initial condition are eliminated to ensure that any slight change will have a greater effect on the following iteration.

The effect of the normalization can be seen in Figs. 5 and 6 which plots 100,000 values of x (initial values: $x = 0.352346764576$, $\alpha = 4.0$) with and without the normalization function, respectively. In Fig. 5, it can be seen that many x values gather around zero and one, whereas Fig. 6 depicts an even distribution after the normalization is applied.

For the second and third stage of each network, the r' parameter is not read from the shared r variable but is instead calculated using a function F as shown in Figs. 2 and 3. This function receives two or four inputs depending on which network is selected and is calculated as

$$r' = 3.86 + (\bar{x} \times 0.14), \tag{3}$$

where $\bar{x}$ is the average of all x inputs to F. The F function ensures that the intermediate r' value remains in the range of [3.86, 4] for optimum chaoticity. After the final stage of each network is completed, the output of the final F function is updated into the shared r variable.

To initialize the network, we set the number of networks being computed in parallel as $\alpha = 8$, the number of stages in each network as $\beta = 3$ and the number of iterations for each local chaotic map $\gamma = 2$. Increasing the value of γ will increase the likelihood that the chaotic maps move randomly to different chaotic trajectories, but will result in lower performance. Each stage of the network has four instances of the chaotic map. Therefore, each random number is produced after a total of $(\alpha \times 4\beta \times \gamma) = 192$ iterations. The selection of α is a trade-off between quality and performance. Experimentally, it was found that eight networks are optimal to harvest sufficient entropy for high-quality random numbers (see Sect. 5.4). Reducing the number of chaotic networks increases the speed of the algorithm but results in reduced entropy, whereas increasing the number of chaotic networks will lower the overall speed of the generator but results in high entropy random numbers. The option of varying the number of networks provides flexibility to a user to customize the generator based on the target application.

Prior to generating the first random number, two chaotic network rounds will be computed to eliminate any transient effects. This ensures that the value of x and r has been sufficiently randomized before the actual random number generation occurs. After the transient elimination rounds, a total of α chaotic networks are computed to produce each one-byte random number. Every instance of the logistic map is iterated γ times. When one thread has finished computing a network, it will be assigned another network until all α networks are completed. The final random number is obtained by mapping the shared x value (floating-point format) into an integer within the range of [0, 255] (to conform to 8-bit ASCII requirements). However, the generator's outputs are in essence, a binary stream. As such, numbers of any size can be obtained by sampling the binary outputs of the generator. The two networks introduce an extra dimension of unpredictability to the proposed TRNG and also indirectly serves as a postprocessing algorithm that ensures that the random numbers are evenly distributed.

In short, the proposed algorithm forces multiple threads to compute multiple chaotic logistic maps that share x and r parameters. The unpredictability of thread access quickly causes the value of x and r diverge from their original values, which in turn makes the selection of networks become arbitrary. The calculation of the intermediary r' value differs for each network where slight variations are reflected by a large change in the output due to the sensitivity of the chaotic map. The entire TRNG algorithm is summarized in Algorithm 1.

Algorithm 1 TRNG Algorithm

Input: Initialized shared **double** x and r, number of networks α, number of stages β, number of iterations γ.
Output: Updated **double** x and r, **unsigned char** R.

```
c ← 0
while c ≠ α do
    //Multiple instances of the while loop is computed in parallel.
    c ← c + 1 //Increment c when a thread is assigned to the loop.
    r' ← r
    for i = 1 to 4 do
        for j = 1 to γ do
            x ← x · r' · (1 − x)
            x ← (x · 1000) − floor(x · 1000)
        end for
        y_i ← 3.86 + (x · 0.14)
    end for
    for k = 2 to β do
        if x ≥ 0.5 then
            //Omega network
            r'_{i={1,2}} ← (y1+y3)/2
            r'_{i={3,4}} ← (y2+y4)/2
        else
            //Clos network.
            r'_{i={1,2,3,4}} ← (y1+y2+y3+y4)/4
        end if
        for i = 1 to 4 do
            for j = 1 to γ do
                x ← x · r'_i · (1 − x)
                x ← (x · 1000) − floor(x · 1000)
            end for
            y_i ← 3.86 + (x · 0.14)
        end for
    end for
    r ← (y1+y2+y3+y4)/4
end while
R ← x · 256 //Final 8-bit random number
```

4 Analysis of Race Conditions as an Entropy Source

Generators proposed in the past have utilized race conditions without first analyzing its effectiveness as a source of entropy [13,15,16,24]. Before delving into the performance of the proposed generator, we will analyze the unpredictability of thread access due to race conditions as a source of randomness. In the first experiment, we observe the sequence of thread access to a particular variable for the quad-core CPU. Each iteration, we store the thread number, $t = [0, 3]$ as 2-bit binary values {00, 01, 10, 11}. We then calculate the average Shannon entropy (SE) of the resulting binary sequence as

$$H(X) = \lim_{k\to\infty} -\frac{1}{k} \sum_{\sigma\in\{0,1\}^k} P(\sigma_k)\log_2 P(\sigma_k), \tag{4}$$

where the summation involves the entire random number sequence represented as binary k-tuples of the form $\sigma_k = \{b_0, \ldots, b_{k-1}\}$, $b_i \in \{0, 1\}$. $P(\sigma_k)$ is the probability to generate k-tuple σ_k, and if $P(\sigma_k) = 0$, then $P(\sigma_k)\log_2 P(\sigma_k)$ is assumed zero. As the sequence consists of 2-bit binary values, $k = 2$. The maximal SE is $H(X) = 1$. Using this methodology, we generate a 1-Gigabit binary sequence that has a SE of $H(X) = 0.9943175$, which is a near-ideal result. We repeat the experiment for the dual-core CPU and obtain a SE of $H(X) = 0.992896$.

Although this preliminary result indicates that race conditions have the potential to be used as an entropy source, Shannon's entropy is not an indicator of non-determinism nor non-periodicity. To measure degree of non-periodicity of a binary sequence, we can calculate its scale index [25]. Scale index is a metric that can be used to quantify non-periodic properties of a data series. It is based on continuous wavelet transform (CWT) and wavelet multiresolution analysis. The

CWT at time u and scale s for a signal f and wavelet function ψ is defined as

$$Wf(u,s):=\langle f,\psi_{u,s}\rangle=\int_{\infty}^{\infty} f(t)\psi_{u,s}^{*}\mathrm{d}t, \tag{5}$$

which defines the frequency component of f corresponding to s and time location, t. The scalogram of f is then calculated as

$$\varsigma(s):=||Wf(u,s)||=\left(\int_{\infty}^{\infty}|Wf(u,s)|^{2}\mathrm{d}u\right)^{\frac{1}{2}}, \tag{6}$$

where $\varsigma(s)\geq 0$ is the energy of a CWT of f at scale s. f has details at scale s if f has a scalogram of $\varsigma(s)>0$. The inner scalogram of f can then be calculated as

$$\varsigma^{\text{inner}}(s):=||Wf(s,u)||_{J(s)}=\left(\int_{c(s)}^{d(s)}|Wf(s,u)|^{2}\mathrm{d}u\right)^{\frac{1}{2}}, \tag{7}$$

where $J(s)=[c(s),d(s)]\subseteq I$ is the maximal subinterval in I for which the support of $\psi_{u,s}$ is included in I for all $u\in J(s)$. Because the length of $J(s)$ is dependent on s, the inner scalogram should be normalized for comparison purposes. This can be calculated as

$$\bar{\varsigma}^{\text{inner}}(s)=\frac{\varsigma^{\text{inner}}(s)}{(d(s)-c(s))^{\frac{1}{2}}} \tag{8}$$

We can then use the normalized inner scalogram for analyzing non-periodicity of a signal, whereby signals that have details at each scale are deemed aperiodic. We then need to select a suitable scale interval, $[s_0,s_1]$ for scalogram analysis, taking into account that the aperiodic behavior of a signal is dependent on its properties at larger scales. To analyze the non-periodicity of thread access due to race conditions, we select a scale interval of [1, 64] similar to [25]. We can then calculate the scale index of f as

$$i_{\text{scale}}:=\frac{\varsigma(s_{\min})}{\varsigma(s_{\max})}, \tag{9}$$

where $s_{\max}$ is the smallest scale where $\varsigma(s)\leq\varsigma(s_{\max})$ for all $s\in[s_0,s_1]$, and $s_{\min}$ is the smallest scale where $\varsigma(s_{\min})\leq\varsigma(s)$ for all $s\in[s_{\max},s_1]$. i_{scale} is thus a value that falls within the range of [0,1], whereby $i_{\text{scale}}\rightarrow 0$ indicates periodicity, whereas $i_{\text{scale}}\rightarrow 1$ indicates highly aperiodic behavior. In the first experiment for the quad-core CPU, the signal f composes of only four possible values, {0, 1, 2, 3}, which corresponds to four threads. Three different signals of 15,000 values were taken for analysis, resulting in scale indexes of 0.3216807, 0.3147504 and 0.2821371. The experiment was repeated for the dual-core CPU which led to scale indexes of 0.1626326, 0.1766252 and 0.1630963. This result shows that the sequence of thread access has some semblance of non-periodicity. In addition, the different scale index values for each dataset also imply that each of them has entirely different sequences despite starting from the same initial conditions. This minor uncertainty will be amplified through the use of chaotic networks, leading to highly non-periodic behavior as shown in Sect. 5.2.1.

5 Security Evaluation

Many TRNG implementations rely on black box statistical test suites such as NIST and DIEHARD as their sole security verification [1,11,26,27]. However, statistical test suites merely detect statistical defects but cannot truly verify the randomness of the source [12]. Despite being deterministic, PRNGs are able to pass these tests. With this in mind, the security evaluation of our TRNG was performed based on the evaluation criteria proposed by Schindler and Killman [12]. Their work was part of the German Federal Office for Information Security (BSI) efforts to standardize the security evaluation of physical TRNGs. Based on this standardization effort, two functionality classes for RNGs were identified, P1 and P2. P1 generators are suitable for applications that require statistical randomness, but the random numbers can be openly transmitted. These generators need to pass statistical test suites to ensure that they are free of statistical defects. Both PRNGs and TRNGs can comply with the requirements of the P1 functionality class.

The P2 functionality class is suitable for high security purposes such as generating encryption keys or signatures. In order to fulfill the P2 requirements, a P1 generator must additionally depict forward and backward unpredictability. Therefore, the security evaluation of our TRNG is divided into two: Firstly, black box statistical testing is performed to ensure that the random numbers generated by our TRNG have good statistical quality. Next, the TRNG is analyzed for non-determinism in terms of random data independence, forward and backward unpredictability as well as data distribution. To analyze the generator's degree of non-periodic behavior, we calculate its scale index.

We also perform additional entropy and nonlinearity analysis to justify the use of the chaotic networks instead of just normalized logistic maps as the entropy extraction method, analyze the effect of changing the number of chaotic networks being computed and depict the distribution of the generator's outputs. As mentioned previously, all experiments are performed on a dual-core (Intel Core2Duo, 2.0 GHz, 3 GB RAM) and quad-core (Intel Core i7-4700MQ, 2.4 GHz, 8 GB RAM) computer.

5.1 Black Box Statistical Testing

The statistical property of the proposed TRNG was evaluated using three well-known statistical test suites, which are the NIST statistical test suite [28], DIEHARD battery of tests [29] and ENT test [30]. The sample sizes for all the statistical test suites have been chosen based on the recommended values obtained from their corresponding documentation and also to match the standard sample sizes used by other TRNG and PRNG proposals for a fair comparison.

The NIST test suite consists of 15 tests performed 1000 times each using different 1-Mbit samples. The passing ratio (PR) and the distribution of P values are calculated for each test after being run 1000 times. The minimum and maximum PR for a significance level of 0.01 is calculated as

$$\mathrm{PR} = 0.99 \pm 3 \times \sqrt{0.99 \times \frac{0.01}{N}}, \tag{10}$$

where N is the number of samples. For all tests, the minimum and maximum PR for 1000 samples is 0.980561 and 0.999439, respectively. The random excursion test is an exception, whereby the 1000 samples are truncated to 625 samples which then has a minimum and maximum PR of 0.9780060 and 1.0, respectively. The proposed generator successfully passes the NIST suite for both dual-core and quad-core versions as shown in Table 1.

The DIEHARD battery of tests consists of 18 tests, each producing a different number of P values. A sequence is considered to have failed a test if a P value falls below 0.01 or close to 1.0 (we set the upper threshold to 0.99). The DIEHARD test suite was performed 50 times with 87.5-Mbit samples, whereby the PR is calculated using Eq. 10, similar to the NIST evaluation. For each individual test which produces multiple P values, each P value is considered as one sample. Therefore, the number of samples differs for each test which leads to varying minimum and maximum PRs as shown in Table 2. Based on these evaluation criteria, generator successfully passed the entire DIEHARD test suite.

The third test suite performed on the proposed generator is the ENT suite which consists of five tests to detect randomness in data. An 80-Mbit sample was used as the input to the ENT suite. The results are as shown in Table 3, along with the corresponding ideal values for each test. Both dual-core and quad-core TRNGs have results that are very close to ideal and therefore has no observable statistical defects.

5.2 Analysis of Non-determinism

To be considered a P2-generator, the proposed TRNG must have backward and forward security in addition to the P1 property. Section 5.1 already shows that our TRNG satisfies the P1 requirement. To prove backward and forward security, we first have to show that the proposed generator is at least Markovian (memory-less and independent). With that in mind, the following notions of an ideal TRNG as provided by Addabbo et al. [31] are observed:

Definition 1 A TRNG with elements from a finite set $\mathcal{M} = \{0, 1, \ldots, m-1\}$ is ideal if and only if it is a sequence of statistically independent and identically distributed (i.i.d) discrete random variables uniformly distributed over m-numbers, and also has no memory of the past generated symbols and no memory of future generated symbols.

Definition 2 A TRNG is unpredictable if and only if it is an ideal TRNG.

To estimate how closely a generator approximates an ideal TRNG, we calculate the average SE of random numbers (Eq. 4). Even a PRNG would be able to achieve a good average SE value as it can approximate a TRNG. In the case of our generator, we wish to depict statistical independence and a memory-less characteristic to prove that it is indeed non-deterministic. With this in mind, the following experiment is conducted [24]: A 1-Megabyte random number sequence is generated by our TRNG with the requirement that the generator is reset after producing each 8-bit number. Resetting the generator returns all the internal states to the default values ensuring that the generated values have no relation to past or future values. A PRNG would immediately fail this requirement as each reset to the same seed will cause the generator to produce the same number indefinitely. To generate a 1-MB sequence, the *generate-reset* procedure is repeated 1,048,576 times. The average SE of the resulting random sequence from our TRNG is calculated to be near-ideal, with $H(X) = 0.999398$ and $H(X) = 0.999978$ for the dual-core and quad-core implementations, respectively.

Next, we can determine whether the TRNG has forward and backward security, whereby an attacker would not be able to identify past or future outputs even if he has the ability to corrupt the internal state of the generator [32]. To fulfill this requirement, evidence has to be provided of an increase in entropy per internal random number that is sufficiently large. This is done by performing the Coron test [33] after the random numbers have shown to be at least Markovian (depicted in the preceding test). The Coron test is executed based on parameters stated in Schindler and Killman's paper, with the length parameter $L = 8$, number of initialization steps, $Q = 2560$, and number of steps, $K = 256{,}000$. For the P2 property to be fulfilled, the test variable has to be $f > 7.976$. Our TRNG attains values of $f = 7.98726$ and $f = 7.99678$ for the dual-core and quad-core implementations, respectively, both fulfilling the predefined criteria.

Unlike the proposed TRNG, the CPU-based generator proposed by Colesa et al. [15] only evaluated their design using one test suite which is the NIST test suite. Their design

Table 1 NIST test results

Test name	2-core *P* value (PR)	4-core *P* value (PR)
Frequency	0.919131 (0.9950)	0.474986 (0.9920)
Block Frequency	0.603841 (0.9890)	0.849708 (0.9890)
Cumulative Sums	0.595549 (0.9870)	0.137282 (0.9870)
Runs	0.163513 (0.9920)	0.639202 (0.9920)
Longest Run	0.616305 (0.9880)	0.597620 (0.9880)
Rank	0.050954 (0.9890)	0.948298 (0.9890)
FFT	0.033807 (0.9890)	0.761719 (0.9890)
Non-overlapping Templates	0.061260 (0.9820)	0.029011 (0.9820)
Overlapping Templates	0.152044 (0.9950)	0.767582 (0.9950)
Universal	0.313041 (0.9860)	0.680755 (0.9860)
Approx. Entropy	0.680755 (0.9940)	0.296834 (0.9940)
Random Excursions	0.055361 (0.9819)	0.060137 (0.9819)
Random Excursions Variant	0.063482 (0.9819)	0.093175 (0.9819)
Serial	0.242986 (0.9870)	0.155499 (0.9870)
Linear Complexity	0.417219 (0.9920)	0.465415 (0.9920)

Table 2 DIEHARD test results

Test name	Samples	Min. PR	Max. PR	PR (Dual)	PR (Quad)
Birthday	500	0.9769	1.0000	0.9900	0.9880
OPERM5	100	0.9608	1.0000	0.9700	0.9700
Binary Rank 31 × 31	50	0.9486	1.0000	1.000	1.000
Binary Rank 32 × 32	50	0.9486	1.0000	0.9800	0.9800
Binary Rank Test 6 × 8	1300	0.9819	0.9981	0.9831	0.9823
Bitstream	1000	0.9808	0.9992	0.9870	0.9830
OPSO	1150	0.9814	0.9986	0.9817	0.9835
OQSO	1400	0.9822	0.9978	0.9864	0.9836
DNA	1550	0.9826	0.9974	0.9832	0.9839
Count-1s Stream	100	0.9608	1.0000	0.9800	0.9700
Count-1s Bytes	1250	0.9817	0.9983	0.9856	0.9824
Parking Lot	550	0.9775	1.0000	0.9782	0.9824
Min. Dist.	50	0.9486	1.0000	0.9600	0.9800
3D-Spheres	1050	0.9810	0.9990	0.9819	0.9829
Squeeze	50	0.9486	1.0000	1.0000	0.9600
Overlapping Sums	550	0.9775	1.0000	0.9836	0.9818
Runs	200	0.9693	1.0000	0.9800	0.9700
Craps	100	0.9608	1.0000	0.9800	0.9900

Table 3 ENT test results

Test name	2-core result	4-core result	Ideal value
Entropy	7.999983	7.999986	8
Chi-square	54.76%	50.95%	50%
Arithmetic mean	127.4911	127.4835	127.5
Monte Carlo	3.142670114	3.141588614	3.141592653
Serial Correlation	−0.000111	0.000053	0

was able to pass the NIST 97.84% of the time with the inclusion of an XOR postprocessing which will effectively reduce their overall throughput by 50%. Another generator proposed by Kim et al. [16] only measured the minimum entropy of their outputs as proof of statistical security. This is equivalent to running the single entropy test within the ENT test suite. Both these proposals still require more statistical testing to determine whether they can even fulfill the P1 requirements. These generators were also not analyzed for non-deterministic properties. The only existing CPU-based TRNG that has fulfilled all requirements of the P1 and P2 functionality classes is the TRNG proposed by Muller [14]. However, the generator suffers from low throughput which will be discussed in Sect. 6.

5.2.1 Degree of Non-periodic Behavior

We additionally measure the degree of non-periodicity of the generator's outputs by calculating their scale index (see Sect. 4 for details). The dual-core and quad-core variants of the TRNG have scale indexes of 0.9064964 and 0.9694791, respectively. Both cases depict values close to 1, which indicates a high degree of non-periodic behavior. To put those values into perspective, we compare them against another true random number generator as well as a pseudorandom number generator. The true random number generator based on bioelectrical and physical signals described in [34] has scale indexes ranging from 0.756 (electrooculography signals as an entropy source) to 0.973 (electroencephalogram signals as an entropy source), whereas the pseudorandom number generator based on quantum walks [35] has scale indexes ranging from approximately 0.35 to 0.7. This comparison shows that the entropy source of the proposed generator is highly non-periodic, as expected from a true random number generator with a non-deterministic entropy source.

5.3 Complexity and Nonlinearity Analysis

In this section, we provide justification for using the chaotic networks for entropy extraction. To do this, we perform complexity and nonlinearity analysis on the outputs of the proposed TRNG and compare it with the outputs of a TRNG based entirely on normalized logistic maps (Eqs. 1 and 2). For the TRNG based on the normalized logistic map, each random number is generated after 1000 iterations of the map in parallel, whereby all threads attempt to read and write to the same shared variable, x. Four different metrics were used for evaluation which are symplectic entropy (SymEn), fuzzy entropy (FuzzyEn), SE and eigenvalue. Experiments were performed on the quad-core CPU.

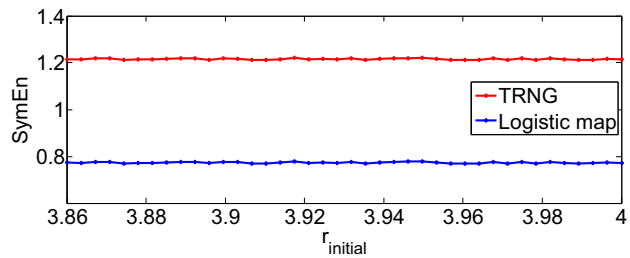

Fig. 7 Symplectic entropy comparison

5.3.1 Symplectic Entropy

A complex system is one which requires a lot of information to describe its behavior. A system can be separated into a series of subsystems, and if each of these subsystems does not depict any identifiable patterns, it implies that the overall system is highly nonlinearity and complex. We choose SymEn as the first metric to evaluate the nonlinearity and complexity of the proposed TRNG under various control parameters in the range of $r = [3.86, 4]$. We follow similar steps in [36,37] to calculate SymEn and plot the results in Fig. 7. The proposed TRNG has uniformly high scores for SymEn throughout its range of control parameters as compared to the TRNG based on the normalized logistic map. A higher SymEn value is a measure of elevated nonlinearity, increased complexity, randomness and resistance to parameter estimation or pattern prediction.

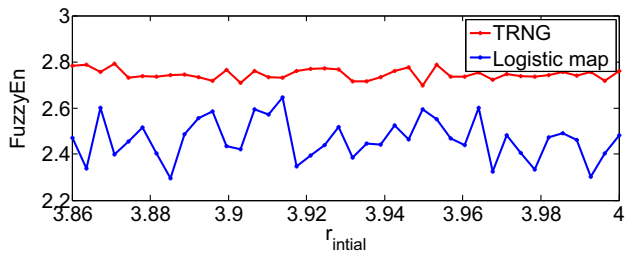

Fig. 8 Fuzzy entropy comparison

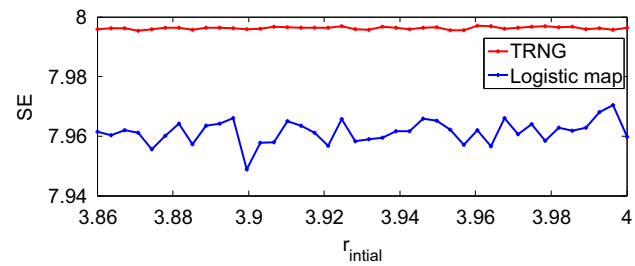

Fig. 9 Shannon entropy comparison

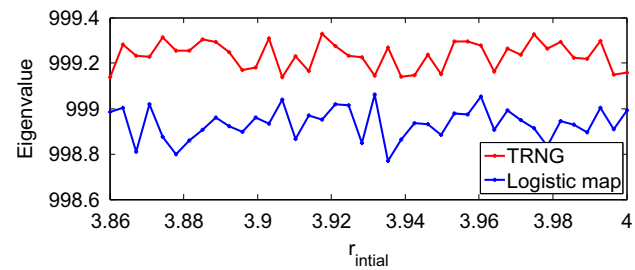

Fig. 10 Eigenvalue complexity ($N = 1000$) comparison

5.3.2 Fuzzy Entropy

FuzzyEn can be used as one of the metrics to assess the degree of uncertainty of a data series. A large FuzzyEn value implies that a particular data series is more complicated as compared to one with lower FuzzyEn values. Based on the steps in [38], we compute the FuzzyEn scores for the proposed TRNG and the logistic map-based TRNG. Rather than the Heaviside function, we utilize a Gaussian membership function where the tolerance value and the embedding dimension are also defined. The Gaussian function used to estimate FuzzyEn is formulated as

$$\Theta(w_{i,j}^m, r) = \exp\left(\frac{-(w_{i,j}^m)^2}{r}\right), \tag{11}$$

where m is embedding dimension, r is the tolerance value, $w = \max_{i,j \in (0,m-1)} |x(i) - x(j)|$ is the maximum distance between two series equivalent to m. Based on varying initial control parameter values, the FuzzyEn results are plotted in Fig. 8. One can see that the FuzzyEn scores of the proposed TRNG have higher values as compared to the TRNG based on the normalized logistic map, which indicates improved complexity.

5.3.3 Shannon Entropy

We also compare both generators based on SE. Data sequences are generated by both TRNGs as 8-bit integers. Thus, the optimal SE value for each 8-bit integer generated is 8 bits per byte. We generate 2^{18} numbers for each value of r being evaluated. Figure 9 shows the SE values of the proposed TRNG and the logistic map-based TRNG, whereby the SE values of data sequences generated by the proposed TRNG have near-ideal SE values as compared to the logistic map-based TRNG. This finding indicates that the outputs of the proposed TRNG are highly random and evenly distributed.

5.3.4 Eigenvalue

Eigenvalue is another metric to evaluate the complexity of a data sequence [39,40]. It depicts the growth rate of phrases and can also effectively assess nonlinear complexity profiles. We select an N-bit sequence and n blocks of symbols to compute the eigenvalue values for each data sequence. Based on the relationship between SE and eigenvalues, the relationship can be mathematically formulated as

$$h = \frac{\log N}{N - E(k)} \tag{12}$$

where h is SE, and $E(k)$ is eigenvalue of the n-block sequence. Figure 10 depicts the eigenvalues of both the proposed TRNG and logistic map-based TRNG for different control parameter values. The eigenvalues for the proposed TRNG are very near to the ideal values for n-block sequences ($n = 2, E(k) = 995.0171, n = 3, E(k) = 996.6781, n = 4, E(k) = 997.5086$) as compared to its logistic map counterpart. This implies the existence of a high level of disorder and complexity.

5.4 Chaotic Network Analysis

In this subsection, the impact of the number of chaotic networks on the quality of random numbers is analyzed. To facilitate this analysis, the following experiment is performed: First, the number of chaotic networks is varied from 1 to 16 and a 1-Mbit sample is produced with the same *generate-reset* method in Sect. 5.2. Next, average SE values are calculated for each of these samples. The experiment is repeated on both the quad-core and dual-core computers and the results are depicted in Fig. 11. From the graph, it can be seen that the quad-core implementation achieves close to maximal entropy when the number of networks computed is

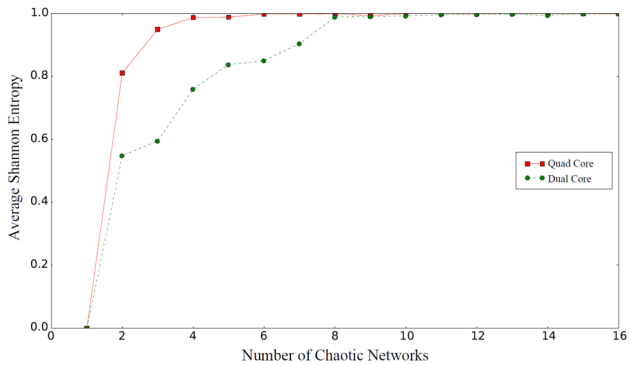

Fig. 11 Entropy analysis of the number of chaotic networks

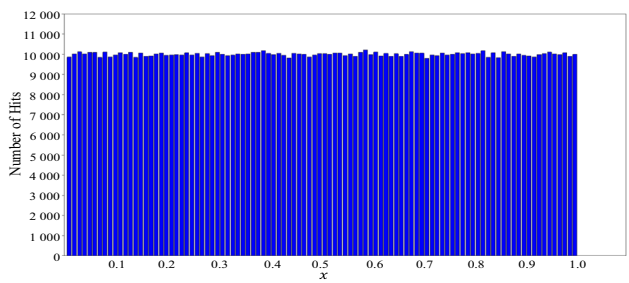

Fig. 12 Dual-core distribution

six, whereas the dual-core implementation requires at least eight. Therefore, eight networks have been chosen as the default setting for the proposed generator to ensure it is applicable even for a minimum of two cores.

5.5 Distribution Analysis

Most applications require PRNGs or TRNGs to be evenly distributed and unbiased. Histogram analysis is used to determine whether our TRNG outputs have this characteristic. A sequence of 100,000 values of x was generated to obtain the histogram plot. The histogram distribution for the dual-core and quad-core variants of the TRNG is as shown in Figs. 12 and 13, respectively. Both versions have uniform distribution, whereby each value of x has an equal probability of occurring.

5.6 Comparison to Other Entropy Sources

In this section, we compare the statistical quality of the proposed TRNG with other TRNGs based on other entropy sources, but limited to widely available computing or wearable devices. We choose the NIST statistical test results for comparison as it is one of the most widely used test suites for random number testing. The results are tabulated in Table 4, where due to space limitations, only the P values of the various TRNGs are compared against the quad-core variant of the proposed generator (all generators have passed the

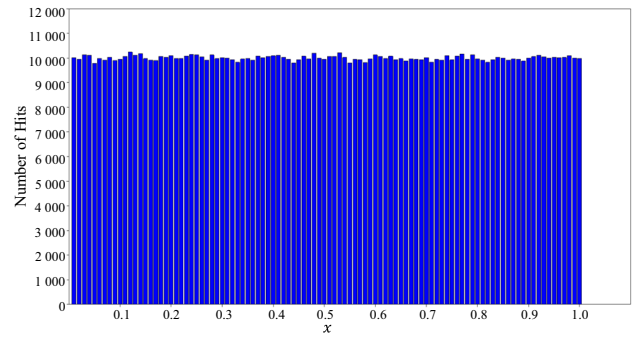

Fig. 13 Quad-core distribution

tests, so the passing ratio is omitted). As the statistical results are dependent on the underlying entropy sources rather than the implementation, we use the results as reported by the TRNGs' respective designers. The results show that the performance of the proposed TRNG is at least on par with the other algorithms, with no significant P value outliers. Thus, it can be concluded that the proposed CPU-based TRNG not only outperforms its peers (those using CPU as an entropy source) but also can perform similarly to other computing device-based state-of-the-art TRNGs.

6 Performance Analysis

The throughput for the proposed generator is 1.3 Mbit/s and 2.2 Mbit/s for the dual-core and quad-core implementations, respectively. The CPU TRNG in [14] records a bit generation speed of 80 Kbit/s on an Intel Core i7 2nd generation which is 16 times slower than the dual-core implementation of our proposed generator.

The generator proposed in [16] did not record any performance data. Additionally, their paper did not provide sufficient information to make a reasonable estimation of their generator's throughput. Although the CPU-based TRNG proposed in [15] also did not record any performance data, a best-case estimation of their generator's throughput can be made. Their generator consists of *steps* that are repeated multiple times to produce a single random bit. Although these *steps* consist of multiple instructions, we will assume that a single *step* only requires one CPU clock cycle for the purpose of estimating the best-case scenario. To produce an output that can pass the NIST statistical test suite 97.84% of the time, a total of 20,000 *steps* are required to generate each bit in addition to an XOR postprocessing. Assuming that the CPU is 2.4 GHz similar to the quad-core CPU used in our work, the throughput of the generator prior to postprocessing is $\frac{2.4\times10^9}{20{,}000\times1024\times1024} = 0.114$ Mbps. The XOR postprocessing halves this throughput to obtain a final value of 0.057 Mbps.

Table 4 NIST comparison of TRNGs

Test Name	CPU (Proposed)	Body Sensors [42]	Mobile Camera [10]	DRAM [43]	GPU [24]
Frequency	0.474986	0.7792	0.516113	0.675	0.444691
Block Frequency	0.849708	0.6787	0.928857	0.096	0.583145
Cumulative Sums	0.137282	0.2974	0.572847	0.588	0.941144
Runs	0.639202	0.2368	0.122325	0.501	0.859637
Longest Run	0.597620	0.7197	0.291091	0.256	0.955835
Rank	0.948298	0.3345	0.530120	0.914	0.146152
FFT	0.761719	0.8831	0.858002	0.424	0.019188
Non-overlapping Templates	0.029011	0.5181	0.743915	0.999	0.431754
Overlapping Templates	0.767582	0.5749	0.502247	0.624	0.062821
Universal	0.680755	0.3838	0.373625	0.999	0.705466
Approx. Entropy	0.296834	0.0909	0.647530	0.735	0.049030
Random Excursions	0.060137	0.6781	0.292960	0.200	0.591876
Random Excursions Variant	0.093175	0.5799	0.966685	0.066	0.276816
Serial	0.155499	0.8188	0.402962	0.405	0.457825
Linear Complexity	0.465415	0.1296	0.433590	0.663	0.991468

This is 22 times slower than the dual-core implementation of our proposed generator at best.

In short, the proposed TRNG in this paper is able to generate random numbers faster than existing CPU-based generators. In addition, it also has sufficient security margins for applications that require truly unpredictable random numbers. Prior CPU-based TRNGs are unable to fulfill both these requirements simultaneously as they either have insufficient security evaluation [15,16] or low efficiency [14]. Therefore, the proposed algorithm can act as a benchmark for future improvements in CPU-based generators.

7 Conclusion

This paper proposed a new TRNG based on multicore CPUs and chaos. The entropy source of the TRNG is the race condition that occurs when multiple threads try to access the same memory location. Chaotic networks were used to amplify any minor uncertainty while simultaneously postprocessing the outputs to eliminate statistical biases. Various portions of the algorithm were designed to change dynamically during run time, which includes selecting between two different chaotic networks and also recalculating control parameters. Both of these portions are also influenced by race conditions, leading to higher uncertainty in the outputs. Statistical tests, entropy, scale index and histogram analysis indicate that the proposed generator has a high degree of randomness, is nondeterministic and has an evenly distributed output. The speed of the generator is also faster than other CPU-based TRNGs in the literature and is scalable with respect to the number of cores within the CPU. Modifications for more speed or randomness are easily performed by adjusting the number of chaotic networks being computed. Multicore CPUs are widely available in modern computers and devices, making the proposed TRNG a cost-effective and easily accessible source of true random sequences for various applications, especially those that require high security. In addition, the proposed work also depicts the viability of chaotic systems when used as postprocessing algorithms for TRNGs.

Acknowledgements This work is supported in part by the Ministry of Education (MOE), Malaysia, under the Fundamental Research Grant Scheme (FRGS), Project Number FRGS/1/2019/ICT05/USM/02/1.

References

1. Cherkaoui, A.; Fischer, V.; Aubert, A.; Fesquet, L.: A self-timed ring based true random number generator. In: 2013 IEEE 19th International Symposium on Asynchronous Circuits and Systems (ASYNC), pp. 99–106 (2013). https://doi.org/10.1109/ASYNC.2013.15
2. Walker, J.: HotBits: Genuine random numbers, generated by radioactive decay. www.fourmilab.ch/hotbits (2001). www.fourmilab.ch/hotbits
3. Kanter, I.; Aviad, Y.; Reidler, I.; Cohen, E.; Rosenbluh, M.: An optical ultrafast random bit generator. Nat. Photonics **4**, 2–5 (2009). https://doi.org/10.1038/NPHOTON.2009.235
4. Liu, Y.; Tang, W.; Guo, H.: True random number generator based on the phase noise of laser. In: 2010 Conference on Lasers and

Electro-Optics (CLEO) and Quantum Electronics and Laser Science Conference (QELS), pp. 1–2 (2010)
5. Pyo, C.; Pae, S.; Lee, G.: Dram as source of randomness. Electron. Lett. **45**(1), 26–27 (2009). https://doi.org/10.1049/el:20091899
6. Antoniadis, K.; Blanchard, P.; Guerraoui, R.; Stainer, J.: The entropy of a distributed computation random number generation from memory interleaving. Distrib. Comput. **31**(5), 389–417 (2017). https://doi.org/10.1007/s00446-017-0311-5
7. Pyo, C.; Lee, G.: Gpus as high-performance random sources. Electron. Lett. **50**(8), 602–604 (2014). https://doi.org/10.1049/el.2013.4047
8. Chan, J.J.M.; Thulasiraman, P.; Thomas, G.; Thulasiram, R.: Stationarity enforcement of accelerator based trng by genetic algorithm. In: 2015 IEEE Trustcom/BigDataSE/ISPA, vol. 1, pp. 1122–1128 (2015). https://doi.org/10.1109/Trustcom.2015.492
9. Teh, J.S.; Teng, W.; Samsudin, A.; Chen, J.: A post-processing method for true random number generators based on hyperchaos with applications in audio-based generators. Front. Comput. Sci. (2019). https://doi.org/10.1007/s11704-019-9120-2
10. Yeoh, W.Z.; Teh, J.S.; Chern, H.R.: A parallelizable chaos-based true random number generator based on mobile device cameras for the android platform. Multimed. Tools Appl. (2018). https://doi.org/10.1007/s11042-018-7015-0
11. Fernández, N.; Quintas, F.; Sánchez, L.; Arias, J.: Social noise: Generating random numbers from twitter streams. Fluct. Noise Lett. **14**(01), 1550,012 (2015). https://doi.org/10.1142/S0219477515500121
12. Schindler, W.; Killmann, W.: Evaluation criteria for true (physical) random number generators used in cryptographic applications. In: Cryptographic Hardware and Embedded Systems—CHES 2002. Lecture Notes in Computer Science, vol. 2523, pp. 431–449. Springer Berlin Heidelberg (2003). https://doi.org/10.1007/3-540-36400-5_31
13. Li, J.; Zheng, J.; Whitlock, P.: Efficient deterministic and non-deterministic pseudorandom number generation. Math. Comput. Simul. **143**, 114–124 (2018). https://doi.org/10.1016/j.matcom.2016.07.011
14. Muller, S.: CPU time jitter based non-physical true random number generator (2014). http://www.chronox.de/jent/doc/CPU-Jitter-NPTRNG.html
15. Colesa, A.; Tudoran, R.; Banescu, S.: Software random number generation based on race conditions. In: 10th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing, 2008. SYNASC'08, pp. 439–444 (2008). https://doi.org/10.1109/SYNASC.2008.36
16. Kim, J.; Yoo, T.; Yeom, Y.; Yi, O.: New Entropy Source for Cryptographic Modules Using OpenMP in Multicore CPUs. International Journal of Security and its Applications **7**(4), 445–452 (2013). http://www.sersc.org/journals/IJSIA/vol7_no4_2013/38.pdf
17. Netzer, R.H.; Miller, B.P.: What are Race Conditions? Tech. rep. University of Wisconsin, Some Issues and Formalizations (1992)
18. Helmbold, D.; McDowell, C.: A taxonomy of race conditions. J. Parallel Distrib. Comput. **33**(2), 159–164 (1996). https://doi.org/10.1006/jpdc.1996.0034
19. Pak, C.; Huang, L.: A new color image encryption using combination of the 1d chaotic map. Signal Process. **138**, 129–137 (2017). https://doi.org/10.1016/j.sigpro.2017.03.011
20. Alawida, M.; Samsudin, A.; Teh, J.S.; Alkhawaldeh, R.S.: A new hybrid digital chaotic system with applications in image encryption. Signal Process. **160**, 45–58 (2019). https://doi.org/10.1016/j.sigpro.2019.02.016
21. Lin, Z.; Yu, S.; Lu, J.: A novel approach for constructing one-way hash function based on a message block controlled 8d hyperchaotic map. Int. J. Bifurc. Chaos **27**(07), 1750,106 (2017). https://doi.org/10.1142/S0218127417501061
22. Avaroğlu, E.; Tuncer, T.; Özer, A.; Ergen, B.; Tűrk, M.: A novel chaos-based post-processing for TRNG. Nonlinear Dyn. (2015). https://doi.org/10.1007/s11071-015-1981-9
23. Akhshani, A.; Akhavan, A.; Mobaraki, A.; Lim, S.C.; Hassan, Z.: Pseudo random number generator based on quantum chaotic map. Commun. Nonlinear Sci. Numer. Simul. **19**(1), 101–111 (2014). https://doi.org/10.1016/j.cnsns.2013.06.017
24. Teh, J.S.; Samsudin, A.; Al-Mazrooie, M.; Akhavan, A.: Gpus and chaos: a new true random number generator. Nonlinear Dyn. (2015). https://doi.org/10.1007/s11071-015-2287-7
25. Benítez, R.; Bolós, V.; Ramírez, M.: A wavelet-based tool for studying non-periodicity. Comput. Math. Appl. **60**(3), 634–641 (2010). https://doi.org/10.1016/j.camwa.2010.05.010
26. Dept, R.B.P.: The Evaluation of Randomness of RPG100 by Using NIST and DIEHARD Tests. Tech. rep. FDK Corporation (2003)
27. Hu, Y.; Liao, X.; Wong, K.; Zhou, Q.: A true random number generator based on mouse movement and chaotic cryptography. Chaos Solitons Fractals **40**(5), 2286–2293 (2009). https://doi.org/10.1016/j.chaos.2007.10.022
28. Rukhin, A.; Soto, J.; Nechvatal, J.: A statistical test suite for random and pseudorandom number generators for cryptographic applications. In: NIST Special Publication 800-22. National Institute of Standards and Technology (2010)
29. Marsaglia, G.: DIEHARD: A battery of tests of Randomness (1996). http://stat.fsu.edu/pub/diehard/
30. Walker, J.: ENT Program (2008). http://www.fourmilab.ch/random
31. Addabbo, T.; Fort, A.; Rocchi, S.; Vignoli, V.: Chaos based generation of true random bits. In: Kocarev, L., Galias, Z., Lian, S. (eds.) Intelligent Computing Based on Chaos, Studies in Computational Intelligence, vol. 184, pp. 355–377. Springer, Berlin (2009). https://doi.org/10.1007/978-3-540-95972-4_17
32. Dodis, Y.; Pointcheval, D.; Ruhault, S.; Vergniaud, D.; Wichs, D.: Security analysis of pseudo-random number generators with input: /dev/random is not robust. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security, CCS'13, pp. 647–658. ACM, New York, NY, USA (2013). https://doi.org/10.1145/2508859.2516653
33. Coron, J.S.: On the security of random sources. In: Public Key Cryptography, Lecture Notes in Computer Science, vol. 1560, pp. 29–42. Springer Berlin Heidelberg (1999). https://doi.org/10.1007/3-540-49162-7_3
34. Tuncer, S.A.; Kaya, T.: True random number generation from bioelectrical and physical signals. Comput. Math. Methods Med. **2018**, 1–11 (2018). https://doi.org/10.1155/2018/3579275
35. Yang, Y.G.; Zhao, Q.Q.: Novel pseudo-random number generator based on quantum random walks. Sci. Rep. (2016). https://doi.org/10.1038/srep20362
36. Lei, M.; Meng, G.; Zhang, W.; Wade, J.; Sarkar, N.: Symplectic entropy as a novel measure for complex systems. Entropy **18**(11), 412 (2016). https://doi.org/10.3390/e18110412
37. Lei, M.; Meng, G.; Dong, G.: Fault detection for vibration signals on rolling bearings based on the symplectic entropy method. Entropy **19**(11), 607 (2017). https://doi.org/10.3390/e19110607
38. Xie, H.B.; Chen, W.T.; He, W.X.; Liu, H.: Complexity analysis of the biomedical signal using fuzzy entropy measurement. Appl. Soft Comput. **11**(2), 2871–2879 (2011). https://doi.org/10.1016/j.asoc.2010.11.020
39. Liu, L.; Miao, S.; Hu, H.; Deng, Y.: On the eigenvalue and shannon's entropy of finite length random sequences. Complexity **21**(2), 154–161 (2014). https://doi.org/10.1002/cplx.21587
40. Alawida, M.; Samsudin, A.; Teh, J.S.; Alshoura, W.H.: Digital cosine chaotic map for cryptographic applications. IEEE Access **7**, 150,609–150,622 (2019). https://doi.org/10.1109/access.2019.2947561
41. May, R.M.: Simple mathematical models with very complicated dynamics. In: The Theory of Chaotic Attractors, pp. 85–93.

Springer New York (2004). https://doi.org/10.1007/978-0-387-21830-4_7
42. Camara, C.; Martín, H.; Peris-Lopez, P.; Aldalaien, M.: Design and analysis of a true random number generator based on GSR signals for body sensor networks. Sensors 19(9), 2033 (2019). https://doi.org/10.3390/s19092033
43. Kim, J.S.; Patel, M.; Hassan, H.; Orosa, L.; Mutlu, O.: D-RaNGe: Using commodity DRAM devices to generate true random numbers with low latency and high throughput. In: 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE (2019). https://doi.org/10.1109/hpca.2019.00011